

Monitoring and Understanding Notebook-Based Learning with Jupyter

A Dashboard for Student Self-Assessment

ARTURO OLIVARES MARTOS, Universidad de Granada, Spain

LISA PAWLOWSKI, University of Duisburg-Essen, Germany

As programming becomes increasingly integrated into scientific and educational workflows, the need to understand how users interact with computational notebooks has grown. This project introduces a JupyterLab dashboard extension that monitors and visualizes notebook activity, including notebook openings, cell execution outcomes, errors, and retries. The system produces summary statistics and visual timelines that portray how the workflow unfolds. The dashboard is intended to be used by students for self-assessment and awareness of their coding patterns. A survey was conducted to gather user feedback on the dashboard's usability and usefulness. The insights gained from this evaluation serve as a pilot study, establishing a baseline for the tool's usability and paving the way for future research on its pedagogical impact.

CCS Concepts: • **Applied computing** → **Interactive learning environments**; • **Human-centered computing** → **Empirical studies in visualization**.

Additional Key Words and Phrases: Jupyter Notebooks, Learning Analytics, Educational Dashboards, Student Self-Assessment, Instructor Insights, Interactive Coding Environments

ACM Reference Format:

Arturo Olivares Martos and Lisa Pawlowski. 2026. Monitoring and Understanding Notebook-Based Learning with Jupyter: A Dashboard for Student Self-Assessment. In *Proceedings of* . ACM, New York, NY, USA, 18 pages.

1 INTRODUCTION

Computational notebooks such as Jupyter have become central tools in scientific computing, teaching, and data analysis. Their interactive structure—mixing narrative text, code, and visual output—makes them highly suitable for inquiry-based learning in programming education [13]. Yet despite their pedagogical advantages, instructors often lack visibility into how students actually work within these environments. Students, on the other hand, often have little opportunity to reflect on their own workflow, error patterns, or problem-solving behavior. Research in programming education consistently shows that learners repeat similar types of mistakes, struggle to identify the sources of their errors, and often adopt inefficient trial-and-error strategies. At the same time, teachers typically only see final code submissions, which provide limited insight into the learners' process [16]. This creates a gap between the correctness of the final code and the path the student took to reach it. Understanding this process is crucial to improve teaching interventions and help students develop their metacognitive skills.

Learning dashboards have been shown to enhance students' self-awareness, foster reflection, and offer instructors actionable insights into learner behavior. However, the majority of existing dashboards focus on high-level learning management system data rather than fine-grained programming behavior within computational notebooks [12]. To bridge this gap, it is first necessary to develop tools that translate raw notebook interactions into meaningful visualizations and, crucially, to evaluate whether students perceive such tools as useful and easy to integrate into their workflow.

Authors' addresses: Arturo Olivares Martos, arturoolivares@correo.ugr.es, arturo.olivares@stud.uni-due.de, Universidad de Granada, Granada, Spain; Lisa Pawlowski, lisa.pawlowski.99@stud.uni-due.de, University of Duisburg-Essen, Duisburg, Germany.

2026. Manuscript submitted to ACM

Manuscript submitted to ACM

This project addresses this gap by designing a JupyterLab dashboard extension capable of tracking detailed notebook interactions, such as cell-level execution histories and error patterns. The primary objective of this study is to evaluate the usability, perceived usefulness, and cognitive workload of this newly developed dashboard from a learner’s perspective. By focusing on the user experience (UX) and technology acceptance, this research aims to determine if the proposed system provides a viable foundation for supporting self-reflection in programming education. To evaluate the system, an exploratory study was conducted with 22 students. Participants engaged with the dashboard while solving Python exercises and subsequently provided feedback through standardized instruments, including the System Usability Scale (SUS), the Technology Acceptance Model (TAM), and the NASA Task Load Index (NASA-TLX) [5, 7, 9]. This approach allows for a comprehensive assessment of whether the dashboard meets the needs of students and identifies potential barriers to its adoption. The findings contribute to the field of learning analytics by providing empirical evidence on the design and acceptance of fine-grained, student-centered reflection tools in computational environments.

2 RELATED WORK

Computational notebooks have gained significant relevance in education and research, and recent work has explored how learners interact within these environments. A growing body of research shows that while notebooks support exploratory coding and flexible workflows, they also introduce challenges for understanding the learning process at a fine-grained level. Systems designed to capture cell-level interaction data, such as execution patterns, time on task and navigation behavior demonstrate that detailed traces can reveal iterative retries, hesitation, and fragmented workflows that remain invisible in final notebook submissions [1]. However, existing approaches of this type are primarily designed for instructor-facing analytics and do not address how such data could support student reflection or metacognitive skill development.

In programming education more broadly, research consistently shows that novice programmers struggle with error identification and debugging, often relying on unsystematic trial-and-error strategies and showing difficulty in locating the source of errors [16]. Additional studies highlight that notebook environments exhibit recurring categories of bugs, suggesting that the structure of notebooks can reinforce certain misunderstandings or error patterns [6]. These findings underline the importance of tools capable of making the coding process more transparent and helping learners understand not only the errors they make, but also how they arrive at those errors. However, for such tools to be effective, they must be seamlessly integrated into the student’s workflow without increasing cognitive burden [3].

Learning analytics dashboards have been widely explored as mechanisms for supporting student awareness, self-regulation, and instructional decision-making. Systematic reviews show that dashboards can enhance metacognitive insight and provide users with actionable feedback on their learning behavior [18]. Despite their potential, most dashboards rely on high-level LMS metrics, such as resource access, assignment completion, or overall activity patterns, which limits their capacity to capture the step-by-step nature of programming work. As a result, these systems offer only coarse-grained representations of learning progress [12].

Research targeting programming courses shows similar limitations. Studies that use notebook-based assignments for analytics tend to track performance at the assignment or submission level, without capturing the underlying cell-level workflow [19]. Other dashboard systems designed to support programming instructors focus on global participation and performance distributions but do not visualize detailed notebook interactions or support student-directed reflection [8]. Together, these approaches illustrate the current focus on high-level, instructor-centered metrics and the scarcity of fine-grained, student-centered analytics for notebook environments.

Across all these lines of research, a consistent gap emerges because existing dashboards rarely provide detailed visualizations of how students work within computational notebooks [1, 20] and even fewer studies investigate the usability and user acceptance of such tools from a student’s perspective. While the theoretical potential for error reduction is high, the practical adoption of these systems depends on whether learners find them intuitive and beneficial rather than a source of additional workload [3].

This gap motivates the present project. While prior research emphasizes the potential of fine-grained analytics for error reduction, the successful implementation of such tools depends heavily on their usability and student acceptance. By developing a student-facing Jupyter dashboard extension, the proposed study contributes empirical evidence on how learners perceive and interact with such visualizations [18]. Unlike existing systems that prioritize instructor monitoring, this work evaluates whether the dashboard is perceived as a useful and low-workload addition to the programming environment [12, 16]. This evaluation of usability and technology acceptance is a necessary first step before the long-term impact on learning outcomes can be effectively measured [3].

3 METHODOLOGY

3.1 Research Design

To investigate the usability, technology acceptance, and perceived cognitive workload of the proposed Jupyter dashboard extension, the study follows a quantitative, exploratory, cross-sectional evaluation design. Using standardized questionnaires, the research assesses students’ initial perceptions of a learner-facing dashboard within a Jupyter-based programming environment. This evaluation serves as a necessary first step to ensure the tool is intuitive and low-burden before its long-term impact on learning outcomes can be tested.

This study focuses on answering the following research questions:

- (1) **RQ1:** How do students evaluate the usability of the learner-facing dashboard?
- (2) **RQ2:** How do students perceive the usefulness and ease of use of the dashboard?

As the research is not intended to test causal effects, it does not employ experimental conditions such as deliberate manipulation of variables or comparisons between control and experimental groups. Accordingly, the data are collected at one point in time only, with no longitudinal measurements taken. The main purpose of this study is to evaluate usability and usefulness of the programmed learner-facing dashboard, while the results may provide a basis for further research. Quantitative data analysis, including the calculation of descriptive statistics and Spearman rank correlations, was performed using IBM SPSS Statistics.

3.2 Dashboard Design

The learner-facing dashboard was designed to provide students with insights into their learning process and to support self-reflection while working in a Jupyter-based programming environment. The dashboard includes features such as visualizations of coding activity, error patterns, and progress over time. It aims to help students identify areas where they may be struggling and to encourage them to reflect on their learning strategies. The dashboard was developed as a JupyterLab extension, allowing it to seamlessly integrate with the existing workflow of students using Jupyter notebooks for programming tasks. The design process has two main components: the data collection mechanism, which captures relevant information about students’ interactions with the Jupyter environment; and the data visualization component, which presents this information in an accessible and informative way.

3.2.1 Data Collection. The data collection mechanism is implemented using JupyterLab’s extension API [14, 15]. From the start of the JupyterLab session, the extension monitors various events such as notebook openings, cell executions, errors, etc. Each event triggers a callback function in the backend Python code, which then stores the relevant information in a SQLite database following a predefined schema. The information collected is organized into different tables:

Users This table stores information about the users, including a personal code and their role, which could be student or teacher. Even though in our project only the student role is relevant, we have included the teacher role for potential future use. Regarding the personal code, it is used to identify users while maintaining their anonymity. This code is only used to link the data collected from the JupyterLab extension to the survey responses, allowing us to analyze the relationship between students’ interactions with the dashboard and their perceptions of its usefulness.

Notebooks This table contains information about the notebooks that students interact with, including the notebook title, its path and, if available, its author.

Notebook Sessions This table captures the sessions of interaction with notebooks. A notebook session is defined as the period of time during which a student has a notebook open; therefore, when the active notebook changes, the previous session is closed and a new one is opened. This table registers which notebook was opened by which user, and the start and end timestamps of the session.

Cells This table stores information about the cells within the notebooks, including the cell type, which can be code, markdown or raw, the cell content at the moment of its creation, the notebook it belongs to, and its Jupyter cell ID.

Active Cells This table captures the active cells during a notebook session. An active cell is the cell that the user is currently interacting with. This table registers which cell was active during which notebook session, and the start and end timestamps of the active period.

Cell Executions This table records the executions of code cells. For each execution, it stores the cell that was executed, the notebook session during which it was executed, the start and end timestamps of the execution, the content of the cell at the moment of execution, and the output of the execution. Regarding the errors, the number of errors and the error message are also stored in this table.

All the data collected and stored in the database provides plenty of possibilities for analysis and visualizations, which can help students to understand their learning process and to identify areas for improvement.

3.2.2 Data Visualization. The data visualization component of the dashboard is designed to present the collected information to the students in an accessible and informative way. The dashboard may be opened using the command palette as a side panel in JupyterLab, allowing students to easily access it while working on their notebooks. The dashboard includes various visualizations, which are intended to help students reflect on their learning process and to identify areas where they may need additional support or practice. A general overview of the dashboard’s design and features is provided in Figure 1. The dashboard is organized into different panels, each of which providing the students with relevant information. In addition to the visualizations, each panel includes a help button that provides a brief explanation of the information being presented and how it can be interpreted. The panels available in the dashboard are:

User Information (Figure 2) This panel displays the user’s personal code and their role.

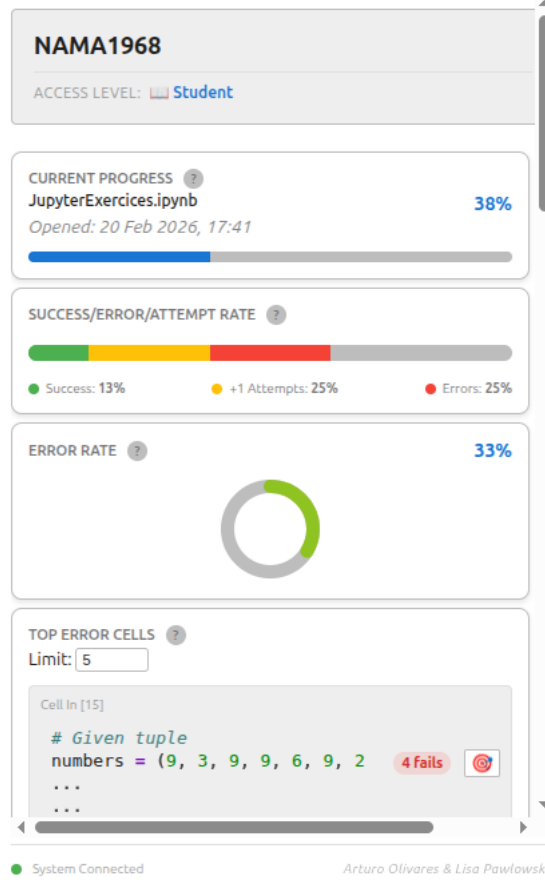


Fig. 1. General overview of the dashboard design.

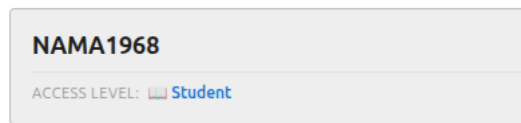


Fig. 2. User information panel.

Current Progress (Figure 3) This panel provides, apart from the title of the notebook and the opening time of the current notebook session, a visualization of the student's progress in the current notebook session. The progress is calculated as described in Equation (1).

$$\text{Progress} = \frac{\text{Number of code cells with an execution without errors}}{\text{Total number of code cells}} \times 100\% \quad (1)$$

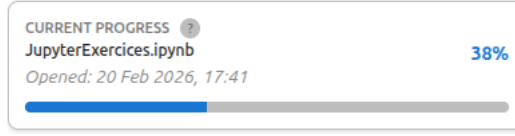


Fig. 3. Current progress panel.

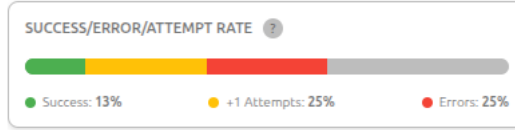


Fig. 4. Success/Error/Attempt rate panel.



Fig. 5. Error rate panel.

Success/Error/Attempt Rate (Figure 4) This panel provides a more detailed view of the student's progress. It classifies the code cells into three categories:

- Successful cells: code cells with only one execution, and that execution did not produce any errors.
- Attempted cells: code cells with more than one execution, having the last execution without errors.
- Error cells: code cells where the last execution produced an error.

The panel shows the rate of successful, attempted and error cells in the current notebook session, calculated as described in the Equation (2).

$$\text{Rate} = \frac{\text{Number of cells in the category}}{\text{Total number of code cells}} \times 100\% \quad (2)$$

Error Rate (Figure 5) This panel focuses on the error rate in the current notebook session, calculated as described in Equation (3). It should be noted that the color used in the visualization changes according to the error rate, being green for 0%, red for 100%, and a gradient of colors in between for the values in between.

$$\text{Error Rate} = \frac{\text{Number of executions with errors}}{\text{Total number of executions}} \times 100\% \quad (3)$$

Top Error Cells (Figure 6) This panel shows the top n code cells with the highest number of executions with errors in the current notebook session. The parameter n can be configured by the user, allowing them to choose how many cells they want to see in this panel. The cells are ordered from the one with the highest number of executions with errors to the one with the lowest. For each cell, the panel displays the cell content of the last



Fig. 6. Top error cells panel.

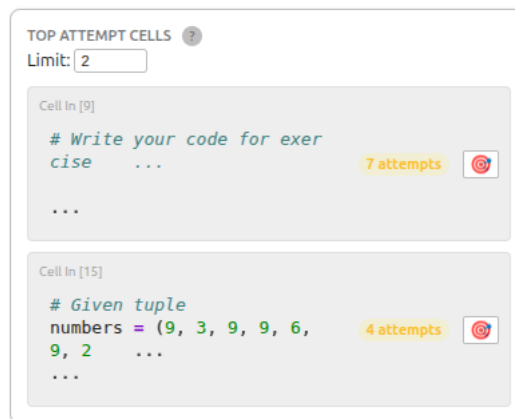


Fig. 7. Top attempt cells panel.

execution, the number of executions with errors, and a button that allows the user to jump to the cell in the notebook.

Top Attempt Cells (Figure 7) This panel shows the top n code cells with the highest number of executions in the current notebook session. For each cell, the panel displays the total number of executions.

Top Time-Consuming Cells (Figure 8) This panel shows the top n code cells with the highest total execution time in the current notebook session. For each cell, the panel displays the total time that the cell has been active during the notebook session.

Top Common Error Types (Figure 9) This panel shows the top n most common Python error types in the current notebook session, such as `IndexError`, `SyntaxError` or `NameError`. For each error type, the panel displays the number of occurrences of that error type.

Execution Timeline (Figure 10) This panel provides a timeline visualization of the code cell executions in the current notebook session. Each execution is represented as a point on the timeline, being the color of the point

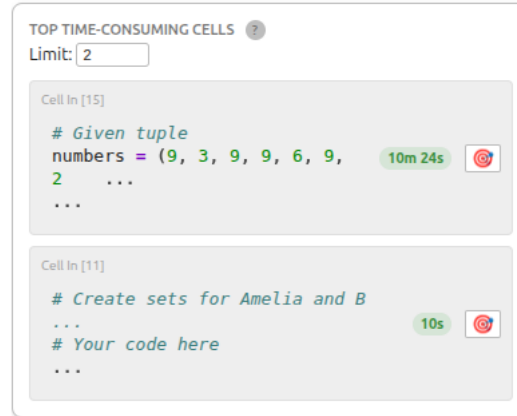


Fig. 8. Top time-consuming cells panel.

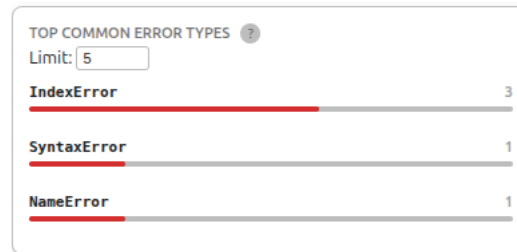


Fig. 9. Top common error types panel.

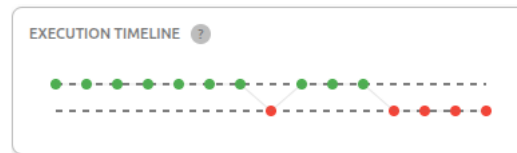


Fig. 10. Execution timeline panel.

green if the execution was successful, and red if it produced an error. This visualization allows students to see the distribution of their successful and unsuccessful executions over time.

3.3 Survey Design

The survey is designed to collect data on students' perceptions of the learner-facing dashboard in a Jupyter-based programming environment.

3.3.1 Data Collection. Before the data collection could start, the study was reviewed and approved by the Ethics Committee of the University of Duisburg-Essen. The ethics approval ID is 2601CMPL2910. The data were collected via an online survey using LimeSurvey, in which participants could take part only once (see Appendix B). This standardized survey design ensured that all participants answered the questions in the same order in order to reduce potential

confounding variables that might arise from differing procedures. In this way, a high level of replicability of the study could be ensured.

At the beginning of the study, participants were informed about the purpose of the study and the associated ethical aspects. It was made clear that participation was voluntary and that participants could withdraw from the study at any time without providing reasons or facing any consequences. Prior to starting the survey, participants were therefore required to give their informed consent. The questionnaire then began with initial questions regarding their previous experience with Jupyter Notebook and Python for comparison purposes. After that, the students began testing the Jupyter Dashboard while solving Python exercises. Subsequently, questionnaires measuring usability and usefulness of the Jupyter Dashboard and a person's subjective mental workload by assessing six key dimensions such as mental demand, physical demand, temporal demand, performance, effort, and frustration level were administered. This was followed by the collection of sociodemographic information, including age, gender, course of study and what semester the participants are in, in order to better describe and analyze the sample. A control question was added at the end to check whether the subjects had completed the questionnaire honestly and carefully. Finally, participants were informed about the further course of the study and thanked for their participation.

3.3.2 Measurements. The usability of the dashboard was assessed using the System Usability Scale (SUS) [7]. The SUS is a standardized and widely used instrument for evaluating the perceived usability of interactive systems. It consists of 10 items that capture users' subjective assessment of system usability, including aspects such as complexity, ease of use, and confidence in using the system. The items are rated on a 5-point Likert scale ranging from 1 ("strongly disagree") to 5 ("strongly agree"). The scale includes five positively worded and five negatively worded items, requiring reverse coding of the negatively formulated statements prior to analysis. A sample item is: "I thought the system was easy to use." The SUS score is calculated by converting item responses according to the standard scoring procedure and multiplying the sum by 2.5, resulting in a total score ranging from 0 to 100, with higher values indicating higher perceived usability. A score above 68 is commonly interpreted as indicating above-average usability. The SUS was chosen because it has demonstrated good reliability and validity across a wide range of domains and technologies with a Cronbach's Alpha = .95 [2].

Perceived usefulness and acceptance of the dashboard were measured using constructs derived from the Technology Acceptance Model (TAM) [5]. According to the model, individuals' intention to use a system is primarily determined by their perceived usefulness and perceived ease of use. Perceived usefulness refers to the extent to which a person believes that using a particular system enhances their performance, whereas perceived ease of use reflects the degree to which a person believes that using the system is free of effort. All items were rated on a 7-point Likert scale ranging from 1 ("extremely unlikely") to 7 ("extremely likely"). Example items include statements such as "Using the dashboard improves my learning performance" and "I find the dashboard easy to use." For each construct, mean scores were calculated by averaging the corresponding items, with higher values indicating stronger agreement. The TAM has been widely validated across technological and educational contexts and typically demonstrates good internal consistency for both dimensions with a Cronbach's Alpha = .89 [11].

Perceived workload associated with using the dashboard was assessed using the NASA Task Load Index [9]. The NASA-TLX is a widely used multidimensional instrument for measuring subjective workload in human-computer interaction and learning environments. It captures the overall perceived task load across six dimensions: mental demand, physical demand, temporal demand, performance, effort, and frustration [9].

Table 1. Sample Characteristics ($N = 22$)

Characteristic	Category / Metric	Value
Gender	Female	9 (40.9%)
	Male	11 (50.0%)
	Unspecified	2 (9.1%)
Age (Years)	Mean (SD)	25.00 (2.15)
	Range	21–28
Coding Knowledge	Mean (SD)	4.82 (2.70)
	Range	0–9
Testing Duration (min)	Mean (SD)	17.29 (7.68)
	Range	3–34

4 RESULTS

4.1 Sample Characteristics

Sample characteristics are available in Table 1. The final sample consisted of 22 students, including 9 female and 11 male participants, while 2 participants did not specify their gender. Participants' ages ranged from 21 to 28 years ($M = 25$, $SD = 2.145$). Thirteen participants reported prior experience with Jupyter Notebook. Self-reported Python proficiency was measured on a scale from 1 to 10 and showed a relatively even distribution across different levels of expertise, allowing the dashboard to be evaluated by students with varying programming backgrounds. The participants spent an average of 17.289 minutes testing the dashboard.

4.2 Reliability of Scales

The internal consistency of the multi-item scales was assessed using Cronbach's alpha [4]. The System Usability Scale demonstrated acceptable to good reliability ($\alpha = .794$). For the Technology Acceptance Model subscales, Perceived Usefulness achieved good reliability ($\alpha = .852$), and Perceived Ease of Use showed excellent reliability ($\alpha = .941$). These reliability estimates confirm that the scales are sufficiently consistent for the purposes of this study, despite the relatively small sample size ($N = 22$). No items were removed to improve alpha, as all contributed positively to their respective scales.

4.3 Descriptive Statistics

Descriptive statistics for all main study variables are presented in Table 2. Participants ($N = 22$) rated the perceived usability of the dashboard with a mean SUS score of $M = 65.57$ ($SD = 17.89$, range 40–90). This value falls slightly below the widely reported average SUS benchmark of 68, indicating usability in the lower-to-average range. The Technology Acceptance Model (TAM) overall score averaged $M = 5.36$ ($SD = 0.88$, range 3.25–6.75) on a 7-point Likert scale. The subscale Perceived Usefulness (PU) had a mean of $M = 4.93$ ($SD = 1.18$), while Perceived Ease of Use (PEOU) was higher at $M = 5.78$ ($SD = 1.06$). These values suggest moderate-to-positive perceptions of usefulness and particularly ease of use. The subjective mental workload (NASA-TLX) yielded a mean overall score of $M = 8.15$ ($SD = 2.92$, range 1.67–12.67) on a scale of 0 to 20. This indicates a relatively low-to-moderate perceived workload [10].

Distributions were largely symmetric with a skewness near 0 for SUS and overall TAM, with some negative skewness in PEOU and NASA-TLX, suggesting a slight tendency toward higher ratings in those scales. Kurtosis values were acceptable overall, with no extreme deviations from normality. To explore the relationships between usability, technology

Table 2. Descriptive Statistics of Main Study Variables ($N = 22$)

Construct	M	SD	Min	Max
System Usability Scale (SUS)	65.57	17.89	40.00	90.00
TAM Overall	5.36	0.88	3.25	6.75
Perceived Usefulness (PU)	4.93	1.18	3.33	7.00
Perceived Ease of Use (PEOU)	5.78	1.06	3.17	7.00
NASA-TLX (Mental Workload)	8.15	2.92	1.67	12.67

Table 3. Spearman Rank Correlations (ρ) Between Main Study Variables ($N = 22$)

Variable	1	2	3	4	5
1. SUS	—				
2. PU (TAM)	.091	—			
3. PEOU (TAM)	.406	.222	—		
4. TAM Overall	.323	.816*	.706*	—	
5. NASA-TLX	-.099	.239	-.255	.024	—

* $p < .001$, so these correlations are statistically significant.

acceptance, and cognitive workload, Spearman rank-order correlations were computed (see Table 3). Given the small sample size ($N = 22$) and the use of Likert-scale data, non-parametric correlations were deemed appropriate. As expected, the TAM subscales Perceived Usefulness ($\rho = .816$, $p < .001$) and Perceived Ease of Use ($\rho = .706$, $p < .001$) exhibited strong, positive, and statistically significant correlations with the overall TAM score. Beyond these internal TAM associations, the analyses did not reveal any statistically significant relationships between the main constructs ($p > .05$). Specifically, perceived cognitive workload (NASA-TLX) did not significantly correlate with overall technology acceptance ($\rho = .024$, $p = .914$) or system usability ($\rho = -.099$, $p = .661$). While there was a moderate positive trend between the System Usability Scale (SUS) and Perceived Ease of Use ($\rho = .406$), this association did not reach statistical significance ($p = .061$).

5 DISCUSSION

5.1 Interpretation of Results

The primary objective of this study was to evaluate the usability, perceived usefulness, and cognitive workload of a student-facing JupyterLab dashboard designed to visualize fine-grained notebook interactions. The results provide a differentiated perspective on how students perceive learner-facing analytics tools in programming contexts and allow the findings to be situated within the broader literature on notebook analytics and learning dashboards.

5.1.1 RQ 1: Usability. The mean SUS score of $M = 65.57$ indicates that the system is marginally acceptable according to established benchmarks. While this score falls just short of the above average threshold of 68, it is important to contextualize this within the internal consistency of the scale $\alpha = .794$. The gap between the observed score and the benchmark suggests that while the core functionality is present, the integration of complex interaction data, such as cell execution histories and error patterns, into a single dashboard may require further UI refinement to reduce perceived complexity. Qualitative feedback from participants explicitly illustrated these interface constraints. For instance, one student suggested that “in top error cells it might be better if it could show error lines in the code and in the dashboard.” This highlights a clear user desire for tighter visual integration between the analytics and the active coding area.

Furthermore, one potential factor contributing to the SUS score falling slightly below the average threshold relates to the dashboard's layout and information density. Due to limited screen real estate, participants were required to scroll vertically to access all visualizations. This necessity for scrolling and searching for information can increase the cognitive burden on users, a challenge that learning analytics tools must actively avoid to be effective [3]. Transitioning to a single-page layout that presents all summary statistics and timelines simultaneously would align better with the established design principle of perceiving learning at a glance [18]. Such a redesign could enhance clarity, reduce navigational effort, and potentially lead to higher technology acceptance and usability ratings.

5.1.2 RQ 2: Usefulness and Ease of Use. A compelling contrast emerged between the TAM subscales. Participants rated the Perceived Ease of Use (PEOU) significantly higher $M = 5.78$ than the Perceived Usefulness (PU) $M = 4.93$. The excellent reliability of the PEOU scale $\alpha = .941$ underscores that the dashboard itself is perceived as intuitive and easy to navigate. These findings strongly align with existing literature on learning analytics. The high PEOU score demonstrates that the dashboard successfully meets the requirement outlined by Corrin and De Barba [3], who emphasize that reflection tools must be seamlessly integrated without increasing cognitive burden. However, the more moderate PU score reflects a well-documented challenge in the field as Corrin and De Barba [3] also note, students often require time and guidance to effectively interpret analytics feedback. While the dashboard visualizes the fragmented workflows and trial-and-error strategies described by Robinson et al. [16], recognizing the metacognitive value of these visualizations as advocated by Schwendimann et al. [18] likely requires longitudinal engagement beyond a 17-minute testing session. Qualitative feedback strongly supported this discrepancy between ease of use and perceived usefulness. Multiple participants expressed that the tool currently feels more aligned with an instructor's perspective, with one student noting, "I feel like the dashboard is more of data analysis rather than for the student itself." Another participant explicitly requested actionable guidance, stating: "I think it would be more useful for a teacher. I think it would be more useful for students to see how to solve a specific error, instead of just getting the feedback which errors you made." This directly echoes existing research on notebook-based analytics tools, which suggests that while fine-grained interaction data can reveal important workflow insights, learners may initially perceive such transparency as descriptive rather than transformative [1, 20]. In other words, students may recognize that the dashboard shows them what they did, but may not yet connect this information to improved future performance without explicit instructional tips. Interestingly, despite these reservations about general usefulness, students did identify specific situational benefits. As one learner pointed out, "I think this dashboard is useful when the learner wants to do a coding exam, so they know how much time they spent on one task," indicating that the perceived value of the tool may depend on the specific educational context.

5.1.3 Overall User Experience and Workload. Although not explicitly bound to a singular research question, assessing the subjective mental workload was a fundamental component in evaluating the dashboard's overall viability. A central and highly positive finding of this study is the low subjective mental workload measured via the NASA Task Load Index (NASA-TLX), which yielded a mean overall score of $M = 8.15$ ($SD = 2.92$) on a 20-point scale. When linearly transformed to a 100-point scale (approx. 40.75), this value sits comfortably below the global median of 43.8 reported in comprehensive user-experience meta-analyses [10]. This result directly addresses a critical concern in the design of educational tools. Computational notebooks already present a dense interactive structure by mixing narrative text, code, and visual output. Introducing an additional layer of fine-grained analytics inherently risks overwhelming the user. However, the low NASA-TLX score strongly suggests that the dashboard fulfills the critical requirement of being a low-workload addition to the programming environment. As emphasized by Corrin and De Barba [3], for learning analytics tools to be genuinely effective, they must be seamlessly integrated into the student's workflow without increasing

cognitive burden. The workload findings confirm that the dashboard successfully avoids this “cognitive burden” trap. It provides students with transparent, cell-level execution histories and visual timelines without distracting them from their primary goal of solving Python exercises. In combination with the high Perceived Ease of Use (PEOU) reported in RQ2, these results indicate that while students may still need pedagogical guidance to translate the dashboard’s descriptive data into actionable steps, the system itself does not impose a significant cognitive hurdle. This lays a robust foundation for future iterations to focus on enhancing the instructional value of usefulness without needing to overhaul the underlying interaction paradigm.

5.2 Limitations

While this study provides valuable initial insights into the usability and acceptance of a learner-facing Jupyter dashboard, several limitations must be acknowledged. First, the sample size of 22 students restricts the generalizability of the findings and the statistical power necessary to detect significant correlations between perceived workload, usability, and technology acceptance. Although the sample size was sufficient for an exploratory evaluation and the scales demonstrated acceptable to excellent internal consistency, a larger and more diverse sample size would be required to draw robust, population-level conclusions. In addition to that, the short duration of the testing phase represents a significant constraint. Participants interacted with the dashboard while solving Python exercises for an average of approximately 17 minutes. As discussed regarding the moderate Perceived Usefulness scores, developing metacognitive skills and recognizing the instructional value of workflow reflection often requires long-term engagement and explicit pedagogical assistance [3]. The brief exposure in this study captures initial impressions but may not accurately reflect how students would perceive, integrate, and benefit from the dashboard over the course of an entire academic semester.

5.3 Future Work

Building upon the foundational usability and technology acceptance findings of this exploratory study, future research should transition from subjective user evaluations to measuring objective pedagogical outcomes. A promising direction for subsequent research would be to conduct controlled experimental studies comparing learners with and without access to the dashboard over multiple learning units. Such studies could investigate whether providing visual feedback on cell execution histories, error rates, and time spent on tasks directly translates into improved programming performance. Researchers could focus on quantifying performance improvements through metrics such as the reduction in error rates between coding sessions, increased execution efficiency e.g., fewer retries per cell, and overall task correctness. To further refine the assessment of execution efficiency, future iterations of the system could utilize specific cell execution durations to evaluate whose code is more computationally optimized. Additionally, to accurately measure overall task correctness, the dashboard should integrate a database of verified solutions. Currently, code that executes without triggering an exception may be interpreted as successful, cross-referencing outputs with a validated database would allow the system to distinguish between code that merely executes without errors and code that is functionally correct. Furthermore, to better understand the mechanisms of self-regulated learning, future iterations could integrate explicit reflection prompts directly into the dashboard interface. By combining system logs with pre- and post-surveys measuring variables like programming self-efficacy and motivation, researchers could determine whether increased metacognitive awareness actively mediates problem-solving success. Finally, while the current project focused exclusively on a student-facing dashboard, future work should expand this concept by developing and evaluating an instructor-facing counterpart. Gathering qualitative and quantitative feedback from educators will be essential to understand how aggregating this fine-grained student interaction data could facilitate targeted pedagogical interventions and reveal common coding

difficulties across an entire cohort. Expanding these analytics to include direct comparisons between different students could provide both instructors and learners with valuable benchmarking insights, highlighting diverse problem-solving approaches and helping educators identify outliers who may need additional support or advanced challenges.

6 CONCLUSION

As programming becomes increasingly integrated into scientific and educational workflows, understanding how students interact with computational notebooks is of crucial importance. This study evaluated a JupyterLab dashboard extension designed to monitor notebook activity and make the iterative, trial-and-error nature of programming transparent to learners. The evaluation demonstrated that while translating complex, cell-level interaction data into an intuitive user interface presents certain layout and usability challenges, the underlying concept is highly viable. Students perceived the dashboard as very easy to use and reported a low subjective mental workload. This is a key success, as it indicates that the system effectively avoids creating additional cognitive hurdles during programming. Although recognizing the immediate instructional usefulness of these visualizations likely requires more longitudinal engagement, targeted pedagogical guidance, and the integration of concrete problem-solving hints, the dashboard lays an important foundation. By shifting away from purely outcome-based evaluations toward process-oriented analytics, tools like this have the potential to sustainably bridge the gap between the correctness of the final code and the students' individual learning paths.

REFERENCES

- [1] Zhenyu Cai, Richard Lee Davis, Raphaël Mariétan, Roland Tormey, and Pierre Dillenbourg. 2025. Jupyter Analytics: A Toolkit for Collecting, Analyzing, and Visualizing Distributed Student Activity in Jupyter Notebooks. In *Proceedings of the 56th ACM Technical Symposium on Computer Science Education V. 1 (SIGCSE TS 2025)*. ACM, Pittsburgh, PA, USA, 1–7. <https://doi.org/10.1145/3641554.3701971>
- [2] Andrew Chu, Dea Biancarelli, Mari-Lynn Drainoni, James H Liu, Jeffrey I Schneider, Ryan Sullivan, and Alexander Y Sheng. 2019. Usability of learning moment: features of an E-learning tool that maximize adoption by students. *Western Journal of Emergency Medicine* 21, 1 (2019), 78.
- [3] Linda Corrin and Paula De Barba. 2014. Exploring students' interpretation of feedback delivered through learning analytics dashboards. (2014).
- [4] Lee J Cronbach. 1951. Coefficient alpha and the internal structure of tests. *psychometrika* 16, 3 (1951), 297–334.
- [5] Fred D Davis et al. 1989. Technology acceptance model: TAM. *Al-Suqri, MN, Al-Aufi, AS: Information seeking behavior and technology adoption* 205, 219 (1989), 5.
- [6] Taijara Loiola de Santana, Paulo Anselmo da Mota Silveira Neto, Eduardo Santana de Almeida, and Iftekhar Ahmed. 2022. Bug Analysis in Jupyter Notebook Projects: An Empirical Study. *arXiv preprint arXiv:2210.06893* (2022).
- [7] German UPA. 2026. System Usability Scale (SUS). <https://germanupa.de/wissen/fragebogenmatrix/sus> Accessed: 2026-02-22.
- [8] Iris Groher, Michael Vierhauser, and Erik Hartl. 2024. A Learning Analytics Dashboard for Improved Learning Outcomes and Diversity in Programming Classes.. In *CSEDU (2)*. 618–625.
- [9] Sandra G Hart and Lowell E Staveland. 1988. Development of NASA-TLX (Task Load Index): Results of empirical and theoretical research. In *Advances in psychology*. Vol. 52. Elsevier, 139–183.
- [10] Morten Hertzum. 2021. Reference values and subscale patterns for the task load index (TLX): a meta-analytic review. *Ergonomics* 64, 7 (2021), 869–878.
- [11] Maslin Masrom. 2007. Technology acceptance model and e-learning. *Technology* 21, 24 (2007), 81.
- [12] Lucas Paulsen and Euan Lindsay. 2024. Learning analytics dashboards are increasingly becoming about learning and not just analytics-A systematic review. *Education and Information Technologies* 29, 11 (2024), 14279–14308.
- [13] Jeffrey M Perkel. 2018. Why Jupyter is data scientists' computational notebook of choice. *Nature* 563, 7732 (2018), 145–147.
- [14] Project Jupyter. 2024. *JupyterLab Extension Tutorial*. Project Jupyter. https://jupyterlab.readthedocs.io/en/v4.5.x/extension/extension_tutorial.html Accessed: 2026-02-22.
- [15] Project Jupyter. 2024. jupyterlab/extension-template: A cookiecutter template for creating a JupyterLab extension. <https://github.com/jupyterlab/extension-template>.
- [16] Derek Robinson, Neil A Ernst, Enrique Larios Vargas, and Margaret-Anne D Storey. 2022. Error identification strategies for Python Jupyter notebooks. In *Proceedings of the 30th IEEE/ACM International Conference on Program Comprehension*. 253–263.
- [17] María Jesús Rodríguez-Triana, Luis P Prieto, Andrii Vozniuk, Mina Shirvani Boroujeni, Beat A Schwendimann, Adrian Holzer, and Denis Gillet. 2017. Monitoring, awareness and reflection in blended technology enhanced learning: a systematic review. *International Journal of Technology Enhanced Learning* 9, 2-3 (2017), 126–150.
- [18] Beat A Schwendimann, Maria Jesus Rodriguez-Triana, Andrii Vozniuk, Luis P Prieto, Mina Shirvani Boroujeni, Adrian Holzer, Denis Gillet, and Pierre Dillenbourg. 2016. Perceiving learning at a glance: A systematic literature review of learning dashboard research. *IEEE transactions on learning technologies* 10, 1 (2016), 30–41.
- [19] Jigneshkumar Jitubhai Sondagar. 2021. *E-Learning - Analytics using Jupyter Notebooks*. Bachelor Thesis. Hochschule Merseburg (FH), Merseburg, Germany. Bachelor Thesis, First Examiner: Prof. Dr. rer. nat. habil Eckhard Liebscher; Second Examiner: Dr. Benjamin Wacker.
- [20] Manuel Valle Torre, Thom van der Velden, Marcus Specht, and Catharine Oertel. 2025. *JELAI: Integrating AI and Learning Analytics in Jupyter Notebooks*. Springer Nature Switzerland, 68–75. https://doi.org/10.1007/978-3-031-98465-5_9

A LEARNING ANALYTICS MODEL

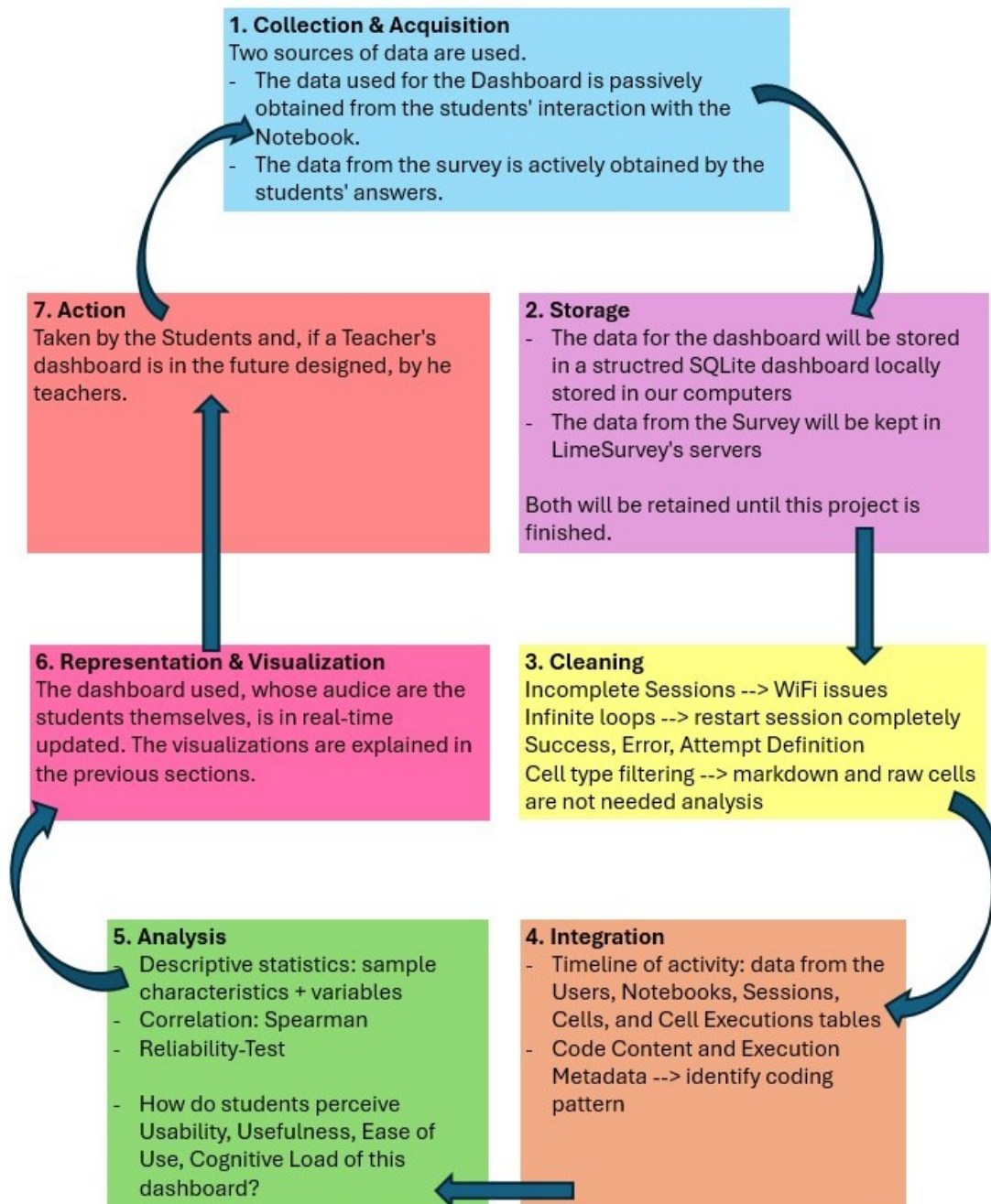


Fig. A.1. Learning Analytics Model for this project.

B QUESTIONNAIRE ITEMS

This appendix provides the exact wording of the items used in the evaluation survey to measure system usability, technology acceptance, and perceived cognitive workload.

B.1 System Usability Scale (SUS)

Participants rated the following 10 items on a 5-point Likert scale (1 = Strongly Disagree to 5 = Strongly Agree).

Table B.1. Items of the System Usability Scale (SUS)

Item	Statement
SUS01	I think I would like to use this Dashboard frequently.
SUS02	I found this Jupyter Dashboard unnecessarily complex.
SUS03	I thought the Jupyter Dashboard was easy to use.
SUS04	I think that I would need the support of a technical person to use the Jupyter Dashboard.
SUS05	I found the various functions in this dashboard were well integrated.
SUS06	I thought there was too much inconsistency in this dashboard.
SUS07	I would imagine that most people would learn to use this dashboard very quickly.
SUS08	I found this dashboard very awkward to use.
SUS09	I felt very confident using this Jupyter Dashboard.
SUS10	I needed to learn a lot of things before I could get going with this dashboard.

B.2 Technology Acceptance Model (TAM)

Participants rated the TAM items on a 7-point Likert scale (Extremely Unlikely to Extremely Likely). The construct was divided into Perceived Usefulness (PU) and Perceived Ease of Use (PEOU).

Table B.2. Items of the Technology Acceptance Model (TAM)

Construct	Statement
PU	<i>Using the Jupyter Dashboard to assist my learning progress would...</i>
PU01	...enable me to accomplish tasks more frequently.
PU02	...improve my learning performance.
PU03	...increase my productivity.
PU04	...enhance my effectiveness during the task.
PU05	...make it easier to do my tasks.
PU06	...be useful.
PEOU	<i>How much do you agree with each statement?</i>
PEOU01	Learning to operate this Dashboard would be easy for me.
PEOU02	I would find it easy to get this Dashboard to do what I want it to do.
PEOU03	My interaction with the Dashboard would be clear and understandable.
PEOU04	I would find this Dashboard would be clear and understandable.
PEOU05	It would be easy for me to become skillful at using this Dashboard.
PEOU06	I would find this Dashboard easy to use.

B.3 NASA Task Load Index (NASA-TLX)

Participants rated the following items on a continuous scale from 0 to 20 to measure their perceived workload during the task.

Table B.3. Items of the NASA-TLX

Dimension	Question
Mental Demand	How mentally demanding was the task?
Physical Demand	How physically demanding was the task?
Temporal Demand	How hurried or rushed was the pace of the task?
Performance	How successful were you in accomplishing what you were asked to do?
Effort	How hard did you have to work to accomplish your level of performance?
Frustration	How insecure, discouraged, irritated, stressed, and annoyed were you?

Received 26 February 2026